Assignment 1: Multi-class Diabetes Risk Prediction

Mohand Sayed (22011902)
Ali Eldeen (22010934)
Youssef Khamis (22011381)

1 Dataset and Preparation

1.1 Data Splitting and Random Seed

To ensure reproducibility, all random operations were initialized with a fixed random seed of 42. The dataset was split into 70% training, 10% validation, and 20% testing using stratified splitting to maintain the original class distribution across all subsets.

1.2 Feature Preprocessing and Selection

Numerical features, specifically BMI, MentHlth, PhysHlth, Education, Income and Age, were standardized using a standard scaler to prevent attributes with larger scales from mathematically dominating distance calculations in models like KNN.

To identify the most predictive features and reduce dimensionality, we applied the Chi-Squared (χ^2) statistical test using a SelectKBest approach. This allowed us to filter the original 21 features down to the 10 most informative attributes, streamlining model efficiency and focusing on the most statistically significant predictors of diabetes risk.

1.3 Handling Class Imbalance

The original dataset suffers from severe class imbalance. To mitigate this during training, we explored three techniques: Random Oversampling, SMOTE (Synthetic Minority Oversampling Technique), and Random Undersampling. Both Random Oversampling and SMOTE successfully balanced the training set by increasing the minority classes to match the majority class, resulting in exactly 133,038 samples per class. SMOTE yielded the best validation performance for our distance-based models.

2 Implementation Details

For each of the following models, two versions were trained: a Baseline version on the original imbalanced training set, and a Balanced version using the handled dataset. Hyperparameters were tuned exclusively using the 10% validation set.

2.1 K-Nearest Neighbors (KNN)

During the hyperparameter tuning phase on the validation set, we experimented with k values of 3, 5, 11, and 21, and evaluated both Euclidean and Manhattan distance metrics. For the Baseline model trained on imbalanced data, the optimal configuration was found to be k=5 using the Manhattan distance. For the Balanced model trained on the SMOTE dataset, the best configuration remained k=5 with Manhattan distance (though Random Oversampling favored Euclidean).

2.2 Softmax Regression

For Softmax (Multinomial Logistic) Regression, the regularization parameter C was tuned over a logarithmic grid: $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100, 10^3, 10^4\}$. The model was trained using the `lbfgs` solver with `multi_class=multinomial` and a maximum of 5000 iterations. The optimal C was selected by maximising the **Macro-F1** score on the validation set, making the selection criterion directly aligned with the primary evaluation metric for this imbalanced dataset.

Beyond the standard balancing strategies (Oversampling, SMOTE, Undersampling, SMOTEENN, and Class Weights), one additional technique was explored:

- **Per-class Threshold Tuning:** Instead of using the default 0.5 decision threshold for all classes, a grid search over thresholds $[0.1, 0.9]$ was performed on the validation set to find the per-class threshold combination that maximises Macro-F1. The best thresholds found were $[0.1, 0.3, 0.3]$ for No Diabetes, Pre-diabetes, and Diabetes respectively.

2.3 Custom Feedforward Neural Network (FNN)

To benchmark a non-linear model, we implemented a custom Feedforward Neural Network (FNN) for multi-class classification. The network was trained on the same preprocessed feature set used by the linear models, and produced class probabilities through a Softmax output layer. The model was optimised with categorical cross-entropy loss and monitored using early stopping on the validation loss to reduce overfitting.

To address class imbalance, the FNN was trained under multiple data strategies: baseline (no balancing), random oversampling, SMOTE, random undersampling, SMOTEENN, and class-weighted loss. Hyperparameters were tuned on the validation set (with a fixed seed of 42 for all randomised operations).

3 Evaluation

3.1 Metric Justification

For this specific imbalanced dataset, **Macro-F1** is the most meaningful and reliable metric. Because the "No Diabetes" class vastly outnumbers the "Prediabetes" class, Accuracy and Micro-F1 are highly misleading; a model can achieve over 80% accuracy simply by guessing the majority class every time. Macro-F1 calculates the F1-score for each class independently and averages them equally, punishing models that completely fail to identify the minority classes.

3.2 Results Summary

Table 1 summarizes the performance of all trained models on the isolated test set.

Table 1: Model Performance Comparison on Test Set

Model Version	**Accuracy**	**Micro-F1**	**Macro-F1**	**Weighted-F1**
KNN (Baseline)	0.8166	0.8166	0.3994	0.7870
KNN (SMOTE)	0.7177	0.7177	0.4166	0.7453
KNN (Oversamp.)	0.7032	0.7032	0.4131	0.7347
KNN (Undersamp.)	0.5827	0.5827	0.3987	0.6693
Softmax (Baseline)	0.8331	0.8331	0.3886	0.7900
Softmax (Oversamp.)	0.6340	0.6340	0.4267	0.7100
Softmax (SMOTE)	0.6384	0.6384	0.4276	0.7100
Softmax (Undersamp.)	0.6893	0.6893	0.4323	0.7300
Softmax (SMOTEENN)	0.5915	0.5915	0.4142	0.6800
Softmax (Class Weights)	0.6501	0.6501	0.4310	0.7200
Softmax (Thr. Tuning)	0.7783	0.7783	0.4602	0.7900
FNN (Baseline)	0.8380	0.8380	0.3915	0.8380
FNN (Oversamp.)	0.6199	0.6199	0.4194	0.6199
FNN (SMOTE)	0.6285	0.6285	0.4186	0.6285
FNN (Undersamp.)	0.6631	0.6631	0.4279	0.6631
FNN (SMOTEENN)	0.5694	0.5694	0.4028	0.5694
FNN (Class Weights)	0.6520	0.6520	0.4243	0.6520

3.3 Confusion Matrices

3.3.1 K-Nearest Neighbors (KNN)

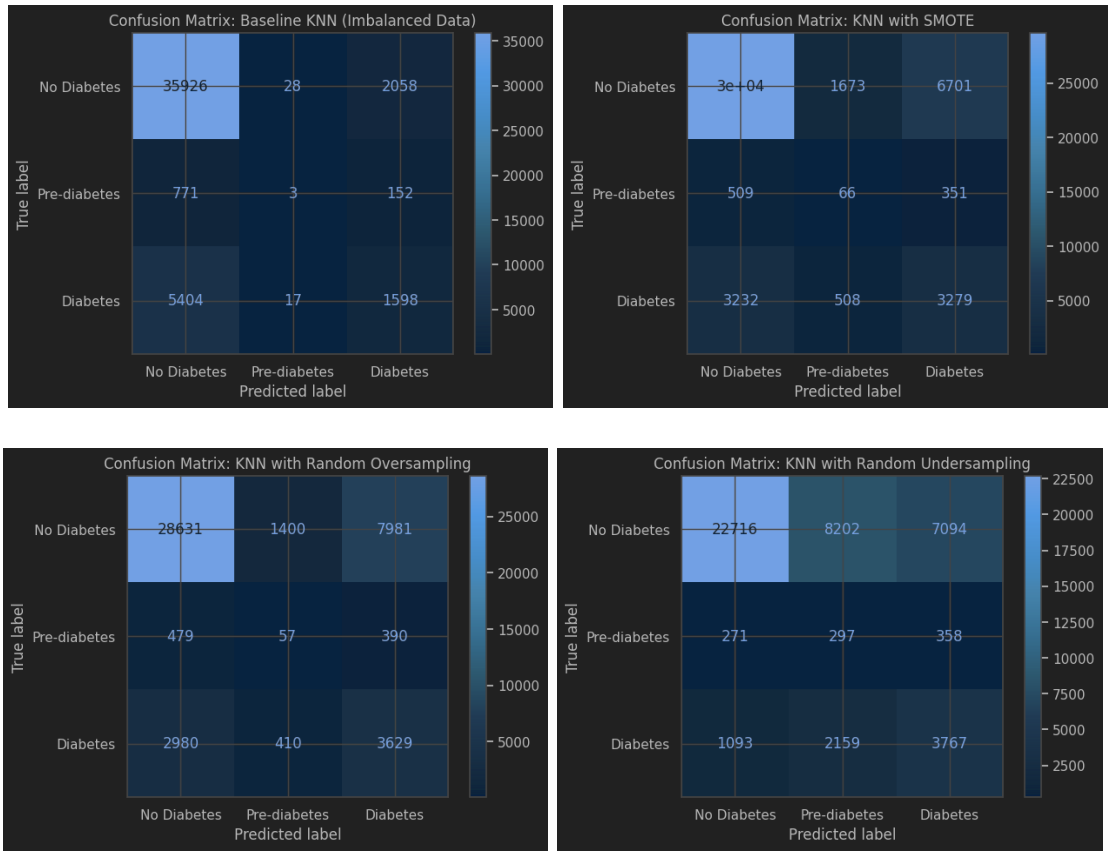

Figure 1: KNN Confusion Matrices (Top Left: Baseline, Top Right: SMOTE, Bottom Left: Oversampled, Bottom Right: Undersampled)

3.3.2 Softmax Regression

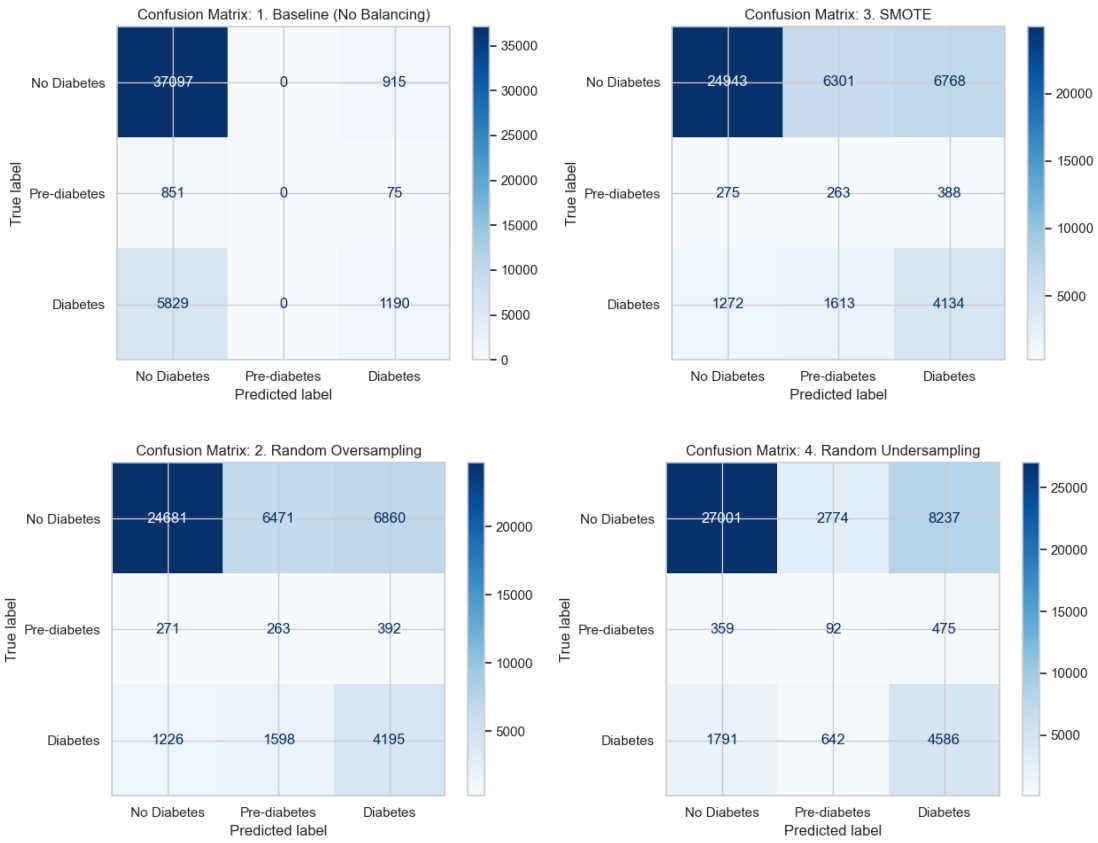

Figure 2: Softmax Confusion Matrices: Baseline (top-left), SMOTE (top-right), Oversampling (bottom-left), Undersampling (bottom-right).

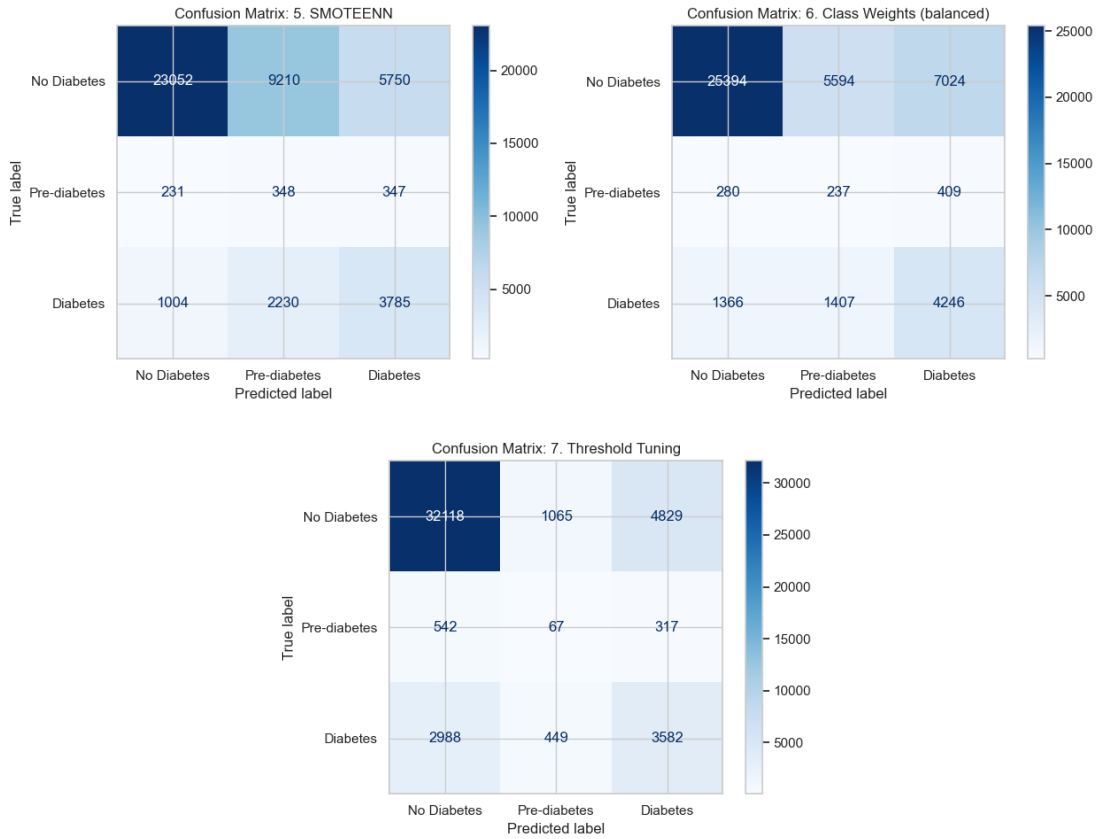

Figure 3: Softmax Confusion Matrices: SMOTEENN (top-left), Class Weights (top-right), Threshold Tuning (bottom-left).

3.3.3 Feedforward Neural Network (FNN)

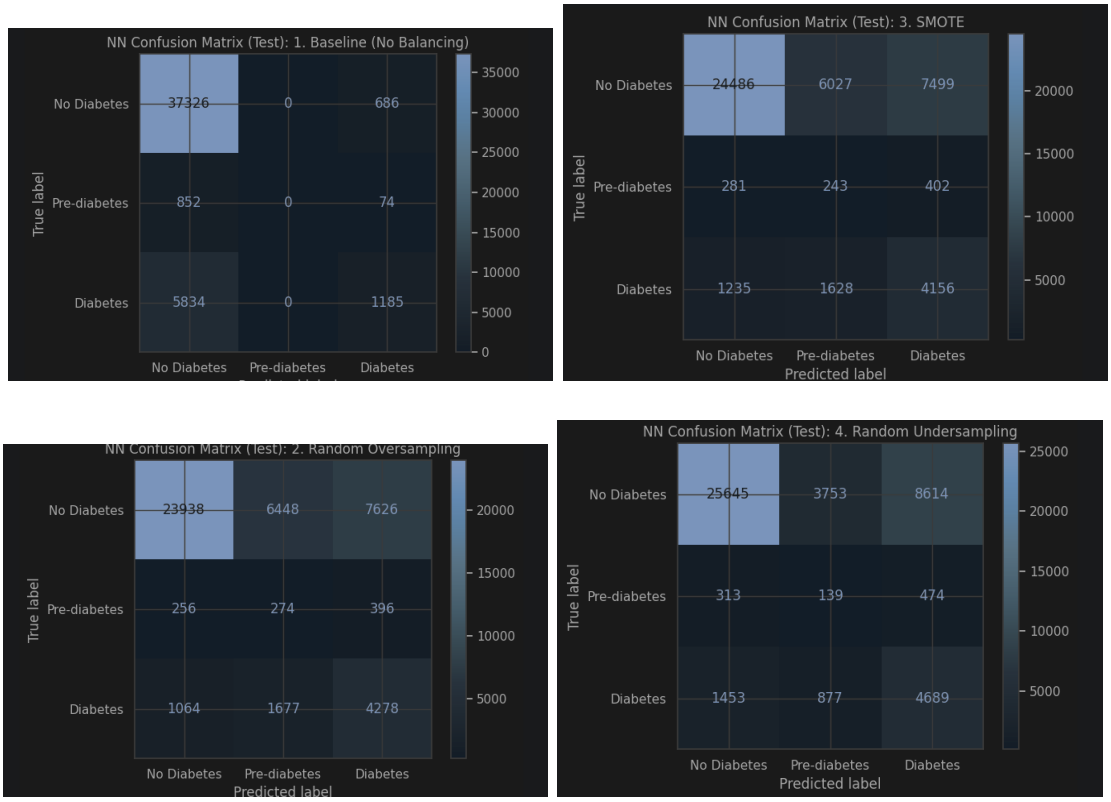

Figure 4: FNN Confusion Matrices: Baseline (top-left), SMOTE (top-right), Oversampling (bottom-left), Undersampling (bottom-right).

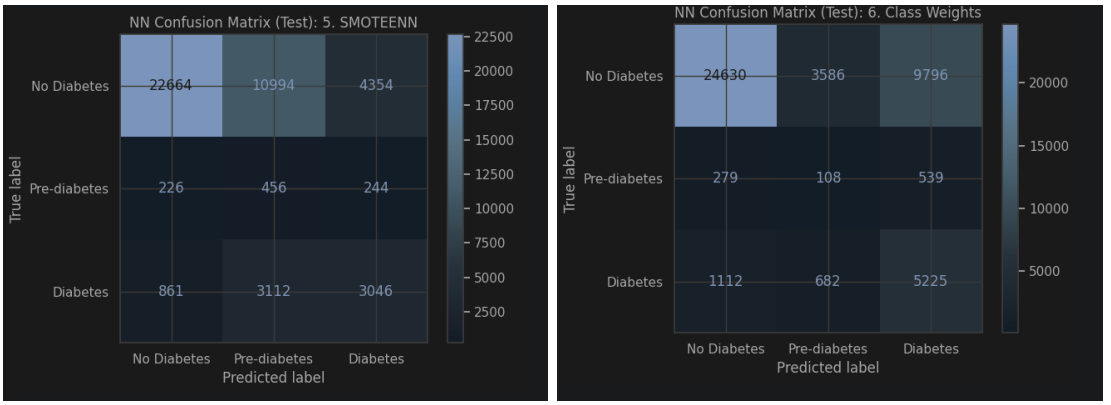

Figure 5: FNN Confusion Matrices: SMOTEENN (left), Class Weights (right).

4 Analysis and Comparison

4.1 Model Comparison

Among the models evaluated so far, the Softmax Regression and KNN models show a clear trade-off between raw accuracy and Macro-F1. The Softmax Baseline achieves the highest accuracy (83.31%) but a low Macro-F1 (0.3886), essentially predicting only the majority

class. Once imbalance handling is applied, Softmax with Threshold Tuning achieves the best Macro-F1 among all Softmax variants (0.4602), outperforming SMOTE-based KNN (0.4166). This demonstrates that a linear model with proper calibration can exceed a distance-based model on this task.

For the FNN, the baseline model again achieves the highest accuracy (0.8380) but the lowest Macro-F1 (0.3915), confirming that accuracy is inflated by the dominant "No Diabetes" class. Once imbalance handling is introduced, the FNN variants converge to a tighter band of Macro-F1 scores, with the best result achieved by Random Undersampling (Macro-F1 = 0.4279). Class Weights (0.4243) yields a very similar performance without altering the dataset size, while SMOTE and Random Oversampling provide comparable improvements (0.4186 and 0.4194 respectively). SMOTEENN performs worst among the balancing strategies (0.4028), likely due to the combined over/under-sampling removing borderline samples that the network could otherwise learn from.

Overall, the FNN improves substantially over its own baseline when trained with imbalance-aware strategies, but it does not exceed the best calibrated linear approach (Softmax with threshold tuning, Macro-F1 = 0.4602). This suggests that, for this dataset, careful decision calibration and imbalance handling can be more impactful than model complexity.

The key insight is that Softmax Regression, being a linear model, is faster to train and tune across many strategies, while KNN is more sensitive to the local geometry of the feature space. Both models struggle significantly with the "Prediabetes" class due to its extreme rarity (only 2% of samples). Threshold Tuning (best thresholds: [0.1, 0.3, 0.3]) partially addresses this by aggressively lowering the decision boundary for minority classes.

4.2 Impact of Imbalance Handling

Comparing the baseline models to the balanced models clearly demonstrates the necessity of handling class imbalance. For KNN, the baseline model achieved a high overall accuracy of 81.66%, but suffered from a low Macro-F1 score of 0.3994, indicating it was heavily biased toward the majority class. Applying SMOTE reduced the overall accuracy to 71.77% but successfully increased the Macro-F1 score to 0.4166. This shift proves that the balancing technique improved the model's ability to predict the difficult minority classes (like Prediabetes), making the classifier much more clinically useful despite the mathematical drop in raw accuracy.

4.3 Insights and Misclassification Patterns

Inspecting the confusion matrices for Softmax Regression reveals a consistent and clinically important pattern: the **"Prediabetes" class is almost entirely misclassified as "No Diabetes"** across all strategies. Even with aggressive balancing (SMOTE, Undersampling, Threshold Tuning), the model rarely correctly identifies a Prediabetes sample.

This occurs for two compounding reasons:

1. **Mathematical:** The Prediabetes class represents only ∼2% of the dataset. Even after balancing the training set, the decision boundary learned by a linear model is dominated by the much larger "No Diabetes" and "Diabetes" clusters. The Prediabetes feature distribution overlaps heavily with "No Diabetes", making it nearly impossible for a linear separator to isolate.

2. **Clinical:** Prediabetes is, by definition, a borderline condition. Patients in this category share many health indicators with healthy individuals (normal BMI, moderate blood pressure), making the distinction genuinely ambiguous even for medical professionals. The features available in this dataset (binary lifestyle indicators) may not capture the subtle biochemical differences that distinguish Prediabetes from No Diabetes.

The Threshold Tuning strategy partially addresses this by lowering the decision threshold for the Prediabetes class, allowing the model to "bet" on Prediabetes more aggressively. However, this comes at the cost of increased false positives for that class.

5 Bonus: TensorBoard Monitoring

During neural network training, TensorBoard was used to monitor convergence and generalisation by tracking both training and validation metrics per epoch. This is particularly important for this dataset because accuracy can look deceptively high due to the dominant "No Diabetes" class; therefore, the validation loss trend is a more reliable indicator of whether the network is learning a stable decision boundary.

5.1 Training Dynamics (Loss and Accuracy Curves)

Figures 5 and 6 show the epoch-level learning curves exported from TensorBoard.

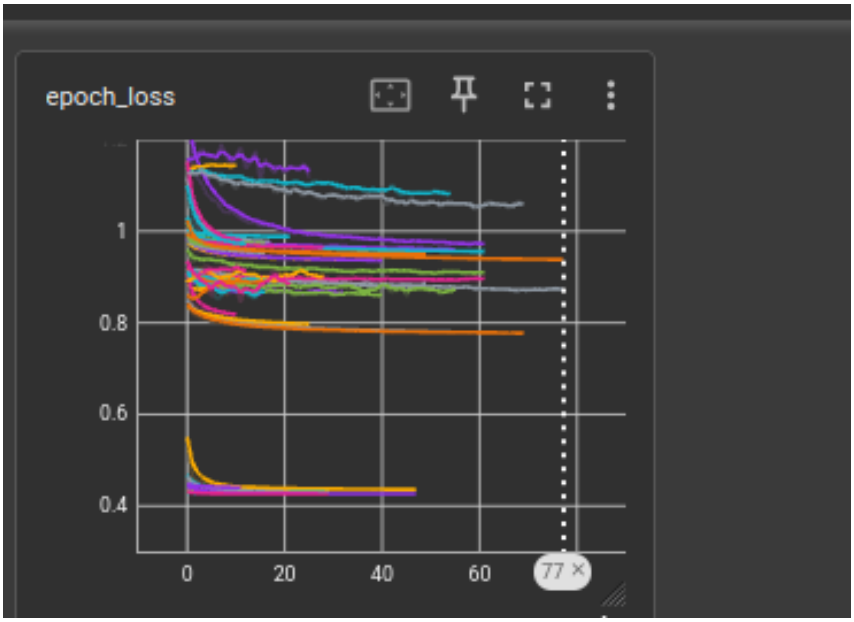

Figure 6: TensorBoard training and validation loss across epochs.

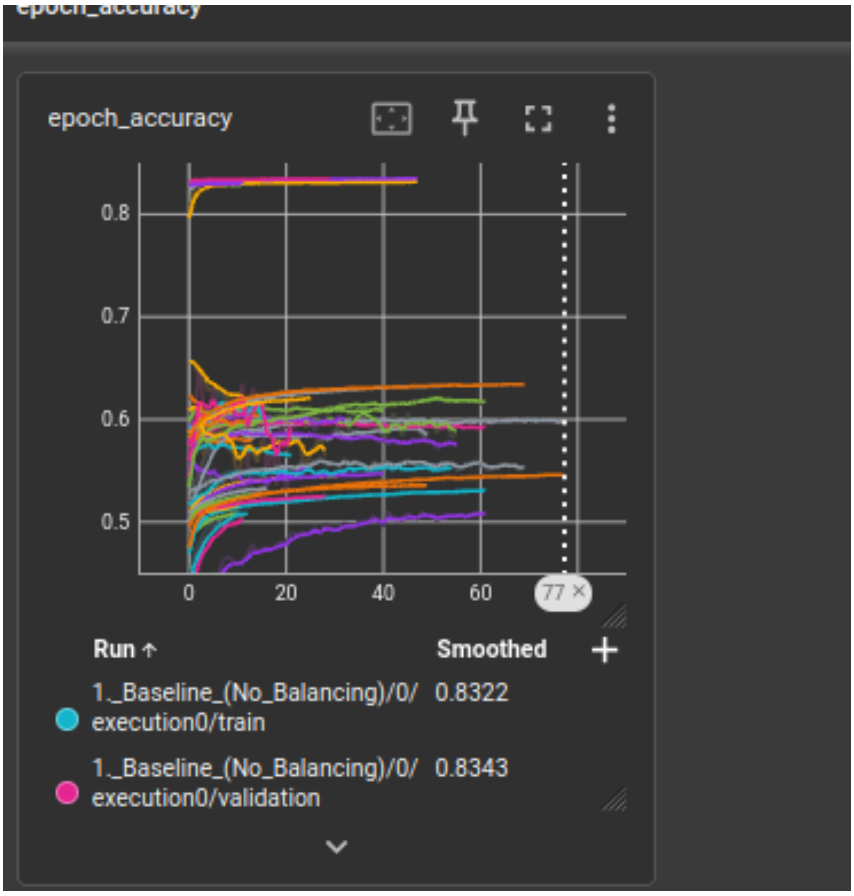

Figure 7: TensorBoard training and validation accuracy across epochs.

5.2 Overfitting vs. Underfitting Diagnosis

We interpret the curves using the following standard criteria:

- **Good fit:** training loss decreases and validation loss decreases (or plateaus) with a small, stable gap.
- **Overfitting:** training loss continues decreasing while validation loss stops improving and begins increasing (a widening generalisation gap). This is often accompanied by training accuracy increasing while validation accuracy stagnates or drops.
- **Underfitting:** both training and validation losses remain high (or decrease very slowly), and accuracies remain low, indicating insufficient model capacity or excessive regularisation.

In our runs, the training loss decreases steadily while the validation loss improves early and then tends to plateau, with only a modest gap between the two curves. This pattern indicates **limited overfitting** rather than severe memorisation. The validation accuracy follows a similar behaviour, improving initially and then flattening, which suggests that the network reaches the limit of what can be learned from the available feature set.

Early stopping (monitoring validation loss) further reduces overfitting risk by restoring the best-performing weights from the point where validation performance peaks. Finally, the remaining discrepancy between training and validation metrics is consistent with the class-imbalance challenge and the difficulty of the minority "Pre-diabetes" class, for which the decision boundary is intrinsically noisy given the feature overlap with "No Diabetes".

5.3 Optimisation Stability (Learning Rate and Internal Statistics)

In addition to loss/accuracy, we exported supplementary TensorBoard plots that help diagnose optimisation stability:

- **Learning Rate:** shows whether the optimiser is training with a constant step size or a schedule.
- **Bias and Beta:** internal parameter statistics from normalisation layers (e.g., BatchNorm), which indicate whether the network parameters are drifting smoothly or changing abruptly.
- **Moving Variance:** tracked by normalisation layers; instability (large oscillations) can indicate noisy gradients or an overly aggressive learning rate.

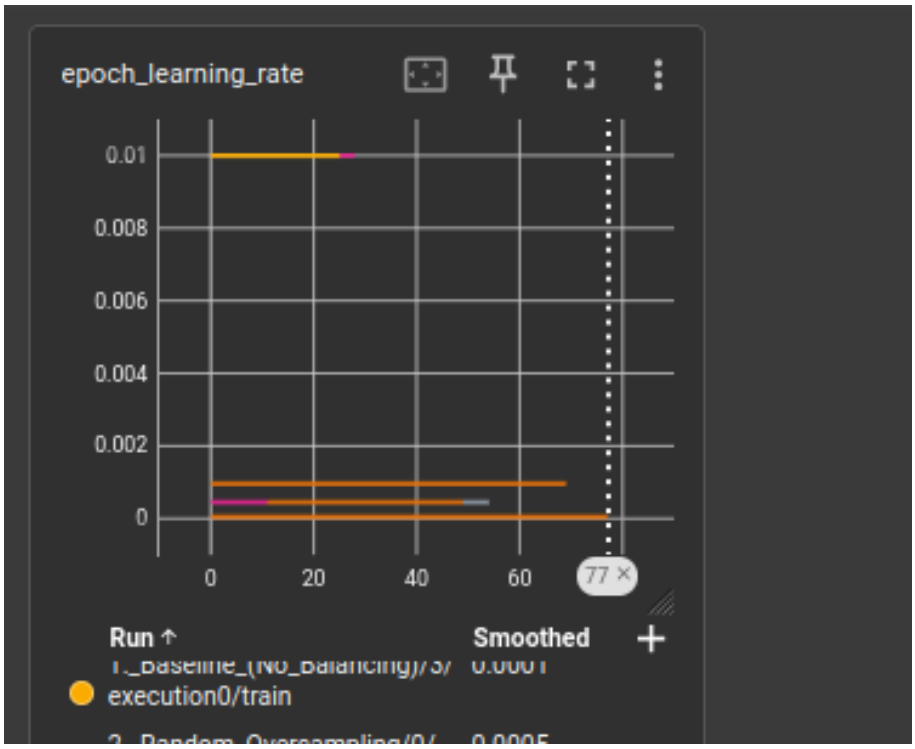

Figure 8: Learning rate over training steps/epochs. A stable (or smoothly decaying) learning rate typically supports steady convergence, while abrupt jumps can correlate with spiky validation curves.

Figure 9: TensorBoard parameter trace (beta). Smooth trends suggest stable optimisation; sharp discontinuities would indicate unstable updates.

Figure 10: TensorBoard parameter trace (bias). When bias parameters grow rapidly while validation loss stops improving, it can be a sign of overfitting or poor calibration.

Figure 11: Moving variance trace from normalisation layers. Stable/slowly-changing variance indicates consistent feature scaling across batches; high oscillation can reflect gradient noise (often worse under heavy undersampling).

Overall, these diagnostic plots suggest that training was **numerically stable**: the learning dynamics evolve smoothly rather than exhibiting abrupt parameter jumps. This supports the conclusion from the loss curves that performance is limited primarily by the dataset difficulty and class overlap (especially for "Pre-diabetes"), rather than by optimiser divergence.